Design Document — Telco Churn Analytics MCP Server

Gil Nussbaum · August 2026 · github.com/Gilnuss/gil-cheq-assignment

Design

An MCP server in Python over the Telco churn dataset (7,043 customers, 52 columns, committed to the repo). The client (Claude Code or Codex) writes the SQL itself: it reads the schema from the server, sends a query, and gets exact results back. Five tools:

- **get_schema** — column types, the real category values, and notes about the data's quirks, so the model doesn't guess wrong values.
- **run_query** — runs a read-only SQL query and returns the result together with the SQL that ran, so every answer can be checked.
- **churn_summary** — ready-made headline stats for overview questions.
- **export_result** — writes a full query result to a CSV file, for when the user wants the raw data itself.
- **sample_rows** — a few example rows.

The server also sends short usage instructions to the client when it connects: read the schema first, aggregate for insights, export for bulk data.

Why

We use DuckDB as the query engine because for this dataset we mostly run aggregations (rates, group-bys, medians). DuckDB queries the CSV files directly so there is no ETL step, and it is built for exactly this kind of analytical workload. It also lets us control which queries are allowed, limit usage and avoid abuse.

Guardrails

The rule: the LLM is not trusted, so every control is enforced below it, never in a prompt.

1. **Engine** — no filesystem or network access, memory capped, configuration locked.
2. **Query** — DuckDB's own parser checks every call: one statement, SELECT only.
3. **Compute** — queries running past 30s are cancelled (configurable). Even on small data, a cross join can create unbounded work.
4. **Response routing** — small results return in the chat; large results are saved as a complete CSV and the chat gets the path plus a preview. The response also reminds the model that if this much raw data was meant to answer a question, a better aggregation is usually the right move. Nothing is trimmed or refused.
5. **Audit** — every call is logged to a DuckDB table that the query connection cannot touch.

Production

- Replace the CSVs and DuckDB with the company warehouse (e.g. Snowflake) as the engine. The tools stay the same; the guardrails move to warehouse features (read-only role, statement timeouts, resource monitors).
- The server becomes a remote service behind SSO, with row and column permissions per user, enforced in the engine.
- With real PII: hash identifiers at ingestion, mask columns by role, block queries that single out individuals.
- Exports go to S3 links instead of local files.
- A test set of questions with known answers runs in CI, so accuracy regressions are caught before users see them.

Risks

- **Wrong but confident SQL** — the biggest risk. Reduced by giving the model the real schema values, returning the SQL with every answer, and the curated summary tool.
- **Malicious queries** — the engine cannot write or reach outside the data; verified with an attack suite.
- **Misreading the data** (e.g. treating a 0/1 flag as money) — covered by the notes in get_schema.
- **Runaway compute or cost** — timeout, memory cap, and response routing.

Business impact at CHEQ

CHEQ produces the same shape of data at scale: traffic verdicts, invalid clicks, account health. Today a question like "which customers spiked in invalid traffic this week?" waits for an analyst. With this pattern, CS managers and security analysts ask in plain language and get exact, checkable answers in minutes. Pointed at per-tenant data with proper permissions, the same server could also become a customer-facing "ask your traffic data" feature.